

CS216 Algorithm Design and Analysis (H)

Greedy / DP / Network Flow 专题练习

题目在前，参考答案在最后

2026 年 6 月

使用建议：

- 每题都按考试大题格式练习：model, algorithm, correctness, complexity。
- Greedy 题重点练 exchange / stays-ahead / lower bound。
- DP 题重点练状态定义、转移、边界、计算顺序。
- Flow 题重点练建图、cut/flow 含义、可行性与整数性。
- 答案在最后，请先独立做完再看。

Part I. Greedy 专题

G1. 最少打点覆盖区间

[10 分]

给定 n 个闭区间 $[s_i, f_i]$ 。请选最少数量的实数点，使每个区间至少包含一个被选点。设计 $O(n \log n)$ 算法并证明正确性。

G2. 最多不重叠区间

[10 分]

给定 n 个区间 $[s_i, f_i]$ ，选择尽可能多的两两不重叠区间。设计 $O(n \log n)$ 算法并证明正确性。

G3. 最少教室安排课程

[10 分]

给定 n 门课的时间区间 $[s_i, f_i]$ 。同一教室不能安排时间重叠的课程。请设计算法求最少需要多少间教室，并给出一种具体安排。

G4. 带截止期的单位作业最大收益

[10 分]

有 n 个作业，每个作业耗时均为 1。作业 i 有截止时间 d_i 和收益 v_i ，若在时间 d_i 之前完成则获得收益，否则不得收益。每个时间单位最多做一个作业。请设计算法最大化总收益。

G5. 最少加油次数

[10 分]

一辆车油箱满油可行驶距离 D 。从起点到终点沿路有加油站，位置为

$$0 = x_0 < x_1 < \cdots < x_n < x_{n+1} = L.$$

每次到加油站可加满油。请设计算法判断能否到达终点，并在能到达时最小化加油次数。

Part II. Dynamic Programming 专题

D1. 加权区间调度

[10 分]

有 n 个任务，每个任务 j 有开始时间 s_j 、结束时间 f_j 和收益 v_j 。选取互不重叠的任务集合，使总收益最大。设计 $O(n \log n)$ 算法。

D2. 0/1 背包

[10 分]

有 n 个物品，物品 i 重量 w_i 、价值 v_i ，背包容量为 W 。每个物品最多选一次。请设计动态规划算法求最大价值，并说明如何恢复被选物品集合。

D3. 最少硬币数

[10 分]

给定硬币面值 $a_1, \dots, a_k$ ，每种硬币数量无限。给定金额 M ，求凑出 M 所需的最少硬币数；若不能凑出，输出 impossible。请给出 DP。

D4. 编辑距离

[10 分]

给定字符串 $X = x_1 \cdots x_m$ 和 $Y = y_1 \cdots y_n$ 。允许 insert、delete、replace，每种操作代价为 1。请设计 DP 求将 X 变为 Y 的最小代价。

D5. 分段最小二乘

[10 分]

给定按 x 坐标排序的点 (x_i, y_i) ，用若干条线段拟合这些点。每开一段有固定费用 C ，每段的误差为该段内点到最佳拟合直线的平方误差和。设 $e(i, j)$ 表示点 $i, \dots, j$ 用一条线段拟合的最小误差。请写出 DP。

Part III. Network Flow / Min-Cut 专题

F1. 二分图最大匹配

[10 分]

给定二分图 $G = (L \cup R, E)$ ，请用最大流算法求最大匹配，并证明建图正确性。

F2. 项目选择与依赖

[10 分]

有若干项目。项目 i 的利润为 p_i ，可能为正也可能为负。若选择项目 i ，则必须选择它依赖的所有项目；依赖关系给出为有向边 (i, j) ，表示选择 i 必须选择 j 。请设计最大化总利润的算法。

F3. 点不相交路径

[10 分]

给定有向图 $G = (V, E)$ 和两个点 s, t 。请设计多项式时间算法求从 s 到 t 的最大点不相交路径条数。除 s, t 外，其余点最多被一条路径使用。

F4. 带上下界的任务分配

[10 分]

有 n 个工人和 m 个任务。工人 i 最少做 a_i 个任务，最多做 b_i 个任务；任务 j 最少需要 L_j 个工人，最多需要 U_j 个工人。若工人 i 能做任务 j ，则给出可行边 (i, j) 。请判断是否存在可行分配。

F5. 软件迁移最大净收益

[10 分]

有 n 个软件。迁移软件 i 可获得收益 $p_i \geq 0$ 。若一对软件 (i, j) 中恰好迁移一个，则产生损失 $d_{ij} \geq 0$ 。软件 1 不能迁移。请设计一个最大化净收益的算法。

STOP

后面是参考答案与解析。

请先独立完成专题练习，再继续往后看。

参考答案与解析

Part I. Greedy 答案

G1 答案

按右端点 f_i 从小到大排序。扫描区间，维护最近选点 x 。若当前区间已包含 x ，跳过；否则选择点 f_i 。

正确性：设当前最早结束且未被覆盖的区间为 $[s, f]$ 。任意可行解必须在该区间中选一个点 y 。把 y 换成 f 不会减少它对之后区间的覆盖能力，因为之后需要考虑的区间右端点都不小于 f 。所以存在包含贪心选择的最优解。归纳即可。复杂度 $O(n \log n)$ 。

G2 答案

按结束时间 f_i 从小到大排序。每次选择与已选区间兼容且结束最早的区间。

正确性用 stays-ahead：令贪心选出的第 r 个区间为 g_r ，任意最优解的第 r 个区间为 o_r 。归纳证明

$$f(g_r) \leq f(o_r).$$

因此若最优解能选 k 个，贪心也不会在 k 之前停止。复杂度 $O(n \log n)$ 。

G3 答案

按开始时间排序。用一个按当前教室结束时间排序的小根堆。扫描课程：

- 若堆顶教室结束时间 $\leq s_i$ ，把该课程放入该教室并更新结束时间。
- 否则新开一间教室。

正确性：若算法开第 d 间教室，说明当前课程开始时已有 $d - 1$ 间教室的课程尚未结束，加上当前课程，共有 d 门课同时重叠，所以任何方案至少需要 d 间教室。贪心使用的教室数达到这个下界。复杂度 $O(n \log n)$ 。

G4 答案

按截止时间 d_i 从小到大排序。维护一个小根堆，存当前决定按时完成的作业收益。扫描作业 i ：

- 将 v_i 加入堆。
- 若堆大小 $> d_i$ ，弹出收益最小的作业。

最后堆中作业为最优选择。

正确性：处理完截止时间不超过 d 的作业后，最多只能按时完成 d 个。若当前选择超过 d 个，保留收益最大的 d 个显然最优；弹出最小收益不会变差。对扫描前缀归纳即可。复杂度 $O(n \log n)$ 。

G5 答案

从当前位置出发，总是开到在距离 D 内能到达的最远加油站；若下一个站都到不了，则 impossible；若终点可达则结束。

正确性：设贪心第 $r + 1$ 次停在最远可达站 g_{r+1} ，任意最优解对应停在 o_{r+1} 。由于 o_{r+1} 从同一位置可达，必有 $g_{r+1} \geq o_{r+1}$ 。把最优解中的 o_{r+1} 换成 g_{r+1} 不会增加停靠次数，也不会破坏后续可行性。交换归纳得贪心最优。复杂度 $O(n)$ 。

Part II. DP 答案

D1 答案

按结束时间排序。定义

$$p(j) = \max\{i < j : f_i \leq s_j\}.$$

状态:

$$M[j] = \text{前 } j \text{ 个任务的最大收益.}$$

转移:

$$M[j] = \max\{M[j-1], v_j + M[p(j)]\}.$$

边界 $M[0] = 0$ 。正确性: 最优解要么不选 j , 要么选 j ; 选 j 时之前只能来自前 $p(j)$ 个任务。排序和二分计算 $p(j)$ 为 $O(n \log n)$, DP 为 $O(n)$ 。

D2 答案

状态:

$$dp[i][w] = \text{只考虑前 } i \text{ 个物品, 容量 } w \text{ 的最大价值.}$$

转移:

$$dp[i][w] = \begin{cases} dp[i-1][w], & w_i > w, \\ \max\{dp[i-1][w], dp[i-1][w-w_i] + v_i\}, & w_i \leq w. \end{cases}$$

边界 $dp[0][w] = 0$ 。答案 $dp[n][W]$ 。恢复方案: 从 (n, W) 倒推, 若 $dp[i][w] = dp[i-1][w]$ 则不选 i ; 否则选 i 并令 $w \leftarrow w - w_i$ 。复杂度 $O(nW)$ 。

D3 答案

状态:

$$dp[x] = \text{凑出金额 } x \text{ 的最少硬币数.}$$

边界 $dp[0] = 0$, 其余初始化为 $+\infty$ 。转移:

$$dp[x] = 1 + \min_{a_i \leq x} dp[x - a_i].$$

按 $x = 1, \dots, M$ 递增计算。若 $dp[M] = +\infty$, 输出 impossible。复杂度 $O(kM)$ 。

D4 答案

状态:

$$dp[i][j] = \text{将 } x_1 \cdots x_i \text{ 变为 } y_1 \cdots y_j \text{ 的最小代价.}$$

边界:

$$dp[i][0] = i, \quad dp[0][j] = j.$$

转移:

$$dp[i][j] = \min \begin{cases} dp[i-1][j] + 1, & \text{delete } x_i, \\ dp[i][j-1] + 1, & \text{insert } y_j, \\ dp[i-1][j-1] + [x_i \neq y_j], & \text{match/replace.} \end{cases}$$

复杂度 $O(mn)$ 。

D5 答案

预处理每个 $e(i, j)$ 。状态:

$M[j]$ = 拟合前 j 个点的最小总代价。

边界 $M[0] = 0$ 。最后一段一定是某个后缀 $i, \dots, j$, 所以

$$M[j] = \min_{1 \leq i \leq j} \{M[i-1] + e(i, j) + C\}.$$

正确性: 枚举最优解最后一段起点 i , 前面部分必须是前 $i-1$ 个点的最优解。若 $e(i, j)$ 已知, DP 为 $O(n^2)$ 。

Part III. Network Flow 答案

F1 答案

建源点 s 、汇点 t 。对每个 $u \in L$ 加边 $s \rightarrow u$ ，容量 1；对每条二分图边 (u, v) 加边 $u \rightarrow v$ ，容量 1；对每个 $v \in R$ 加边 $v \rightarrow t$ ，容量 1。求最大流，所有流量为 1 的 $L \rightarrow R$ 边构成匹配。

正确性：容量限制保证每个左点和右点最多匹配一次。任意匹配对应同等大小整数流；任意整数流对应同等大小匹配。整数容量保证存在整数最大流。复杂度为一次最大流。

F2 答案

这是最大权闭合图。建图：若 $p_i > 0$ ，加边 $s \rightarrow i$ 容量 p_i ；若 $p_i < 0$ ，加边 $i \rightarrow t$ 容量 $-p_i$ ；对依赖边 (i, j) 加边 $i \rightarrow j$ 容量 M ，其中 M 足够大。求最小割，取源侧点集 S 。

若 $i \in S$ 但依赖的 $j \notin S$ ，割会切容量 M 的边，不可能最优，所以 S 满足依赖闭合。割容量等于未选正利润损失加已选负利润代价，因此最小割等价于最大总利润。

F3 答案

拆点。对每个 $v \neq s, t$ ，拆成 v_{in} 和 v_{out} ，加边 $v_{in} \rightarrow v_{out}$ 容量 1。对 s, t 可设容量无穷或不拆。原边 (u, v) 改为 $u_{out} \rightarrow v_{in}$ ，容量无穷；涉及 s, t 时用对应源汇端点。求 s 到 t 最大流。

每个中间点的拆点边容量 1，保证最多一条路径使用该点。整数流可分解为点不相交路径。最大流值即最大条数。

F4 答案

建带下界网络。源点 s 到工人 u_i 加边容量 $[a_i, b_i]$ ；若工人 i 能做任务 j ，加边 $u_i \rightarrow v_j$ 容量 $[0, 1]$ ；任务 v_j 到汇点 t 加边 $[L_j, U_j]$ ；再加边 $t \rightarrow s$ 容量 $[0, \infty]$ 转成 circulation。

处理下界：每条边先送下界 ℓ_e ，剩余容量 $u_e - \ell_e$ 。计算

$$b(v) = \sum_{in} \ell_e - \sum_{out} \ell_e.$$

若 $b(v) > 0$ 加 $v \rightarrow TT$ 容量 $b(v)$ ；若 $b(v) < 0$ 加 $SS \rightarrow v$ 容量 $-b(v)$ 。若最大流能饱和所有 SS 出边，则可行。流量为 1 的工人到任务边即分配。

F5 答案

令 S 为迁移的软件集合，且 $1 \notin S$ 。目标：

$$\max_S \left(\sum_{i \in S} p_i - \sum_{\text{split}(i,j)} d_{ij} \right).$$

建最小割图：对每个软件加点。加边 $s \rightarrow i$ 容量 p_i 。对每对有关联的软件 (i, j) ，加无向惩罚边，可用两条有向边 $i \rightarrow j$ 、 $j \rightarrow i$ ，容量 d_{ij} 。为强制软件 1 不迁移，加边 $1 \rightarrow t$ 容量 M ， M 足够大。

取最小割源侧为迁移集合。若正收益点不在源侧，会切掉 $s \rightarrow i$ ，表示放弃收益；若一对软件分居两侧，会切惩罚边，表示支付损失；软件 1 若在源侧会切无限边，因此不会被选。最小化割容量等价于最大化净收益。